

② AI-Driven School 第3回講義

面倒な報告を自動化する

毎日・毎週の報告をAIで自動化する方法。
その仕組みと、品質を上げる3つのコツ——

自動化ツール = 4つのパーツの組み合わせ

レストランに例えると、注文 → 冷蔵庫 → キッチン → テーブル



実際のツール例

🐱 コミネコ（コミットレポート）

📊 進捗可視化ダッシュボード

💬 LINE朝活レポート

ここから先で、仕組みとコツをひとつずつ解説していきます。

💡 なぜ報告の自動化が「人生を変える」のか

報告を自動化すると「作業時間が浮いて楽になる」。それだけだと思いませんか？ 実は、報告の自動化には**もっと大きな価値**が隠れています。

仕事には**認知 → 意思決定 → 実行**という3つのステップがあります。多くの人は「実行」にエネルギーを注ぎますが、最も力を入れるべきなのは、実は一番上流の**「認知」**です。

仕事の3ステップ

上流を変えれば、その先がすべて変わる



🍴 飲食店で考えてみよう

- 👁 **認知:** 今月の売上が先月より15%落ちていると知る
- 🧠 **意思決定:** ランチメニューを見直そうと決める
- 🔧 **実行:** 実際にメニューを作り変える

⚠ 売上が落ちていることに気づかなければ、メニューを見直そうという判断自体が生まれない

👁 見えやすい効果

- ✓ 作業時間が減る
- ✓ 手間がなくなる
- ✓ 報告の抜け漏れがなくなる

🔍 見えにくい効果（本当の価値）

- 🛡 **起きなかった事故・炎上を防ぐ**
- ➡ **取り逃がさなかった売上を守る**
- 💡 **新しいアイデアをひらめくチャンスが増える**

💡 ここがポイント

報告の自動化の価値を「面倒な作業が減った」だけで判断しないこと。**認知が早くなれば意思決定が変わり、行動が変わり、結果が変わる。**目に見える効果より、見えない効果の方がはるかに大きい。

🧩 自動化ツールの4パーツ

毎日・毎週レポートを作ってくれる自動化ツールには、**必ず4つのパーツ**があります。どんなツールも分解するとこの4つです。料理に例えるとわかりやすくなります。



トリガー = 注文

ツールを動かすきっかけ

「いつ」「何をきっかけに」ツールが動くかの設定。トリガーなしに勝手に動き出すことはない。

🕒 毎朝7時

✎ ボタンを押したら

📎 ファイルが更新されたら

💬 チャットで発言したら



ソース元 = 冷蔵庫

データを取ってくる場所

冷蔵庫には新鮮な食材が必要のように、ソース元にも**新鮮なデータ**がなければ取ってくる意味がない。

📁 Gitリポジトリ

📊 スプレッドシート

☑ タスク管理ツール

✉ メール受信箱



処理する場所 = キッチン

プログラムを実行するサーバー

料理にキッチンが必要のように、プログラムにも「動かす場所」が必要。どこを選ぶかで使える道具やコストが変わる。

📁 GitHub

📊 GAS

△ Vercel

**届ける先 = テーブル**

完成した報告を届ける場所

Slack

LINE

Teams

メール

パブリックURL

**実際に動いている2つのツール**

講義で紹介された2つの報告自動化ツールを、4パーツに分解して見てみましょう。

**コミネコ**

コミットレポートツール

誰がどんな作業をしたのか、Gitのコミット履歴をAIが分析し、ビジュアルなレポートを毎朝自動で届けてくれる。

- 🕒 **トリガー:** GitHub Actions（毎朝7時）
- 📖 **ソース元:** GitHubリポジトリのコミット
- 🔧 **処理:** AIが主導 → 図解HTML → キャプチャ
- 📩 **届ける先:** Slack（画像で投稿）

**進捗可視化ツール**

プロジェクト進捗ダッシュボード

タスク管理ツールから進捗を取得し、信号機の色（青・黄・赤）で一目で状況がわかるダッシュボードを自動生成する。

- 🕒 **トリガー:** GitHub Actions（スケジュール）
- 📖 **ソース元:** Linear（タスク管理ツール）
- 🔧 **処理:** プログラムが主導 + AIは提案文のみ
- 📩 **届ける先:** Slack（画像で投稿）

AI主導 vs プログラム主導 —— 使い分けの考え方

✦ AI主導

- ✓ 柔軟な仕事ができる
- ✓ 難しいデータをわかりやすい言葉に変換
- ✗ 出力がブレる・たまに壊れる
- ✗ 動かした分だけ費用がかかる

＜＞ プログラム主導

- ✓ 同じことを正確に繰り返せる
- ✓ 高速で費用がかからない
- ✗ 柔軟な判断はできない
- ✗ 想定外のデータに弱い

ベストな答え: 両方を組み合わせる。プログラムで確実にできることはプログラムに、AIにしかできないことだけAIに。

🔑 APIキーという「鍵」を理解する

自動化ツールは他のサービスからデータを取ってきたり、チャットに投稿したりします。しかし、誰でも自由にアクセスできたら危険です。そこで必要になるのが**APIキー**（＝サービスにアクセスするための「鍵」）です。

APIキー = パスワードのようなもの



鍵があれば

サービスのデータに自由にアクセスできる



流出すると

他人にカレンダーを見られたりAIを勝手に使われたりする



安全に管理

GitHubの「Secrets」など安全な場所に保管する



絶対にやってはいけないこと: APIキーをGitにコミットすること。一度でもコミットすると、後で消しても履歴に残る。AIにAPIキーをそのまま渡すのもセキュリティ的によくない。

🎯 報告を自動化する3つのコツ

01 完成形の絵から作る

📖 AI-Driven Rules #8 「抽象指示より模範解答を作れ」

最初にやるべきことは仕組みを作るのではなく、**最終的にどんな報告の絵が届けばいいのか**を決めること。ゴールの見た目を先に決めれば、それがAIへの正確な指示になる。

👍 見た目を作る時のポイント

- 🖥️ **Tailwind CSS**を使う（「モダンなフレームワークで」と伝えればOK）
- ⚡ **パッと見て2秒**で何が言いたいかわかるように
- 📷 **画像で送るのがおすすめ**（リンクだと1タップ多い）
- 📱 **スマホファースト**で作る（外出先でも確認できるように）

02 専門家（サブエージェント）に磨かせる

📖 AIの3つの壁を越えるための技術

AIに作らせたレポートを、**別のAI（サブエージェント）にレビューさせる**。同じAIに「これ問題ない？」と聞いても、自分の仕事を正しいとする傾向がある（一貫性バイアス）。頭がフレッシュな別のAIにレビューさせることで、的確な指摘を受けられる。

サブエージェントの仕組み



⚠️ **注意:** サブエージェントには**レビューだけ**に集中させること。修正作業は行わせない。レビューと修正を同時にやらせると、アテンション（注意力）が分散して性能が落ちる。

03 ブレをコードで止める

□ AIの限界を超える人 = AIとコードの使い分けがわかる人

AIの出力はどうしてもブレる。月曜日は正しく出てきたのに水曜日は壊れた、ということが起きる。**プログラム（コード）でAIの弱点をカバーするのが、品質安定の鍵。**



1. チェックする（バリデーション）

メールアドレスの「@マーク」チェックと同じ。AIの出力を**プログラムで機械的に検証**する。形式は正しい？ 必要な項目は全部入っている？ 想定外の値はない？



2. 型にはめる（テンプレート）

たい焼き機のように、**毎回同じ形のHTML/CSSのテンプレートを用意**して、AIが作った文章をそこに流し込む。「模範解答を見て作って」ではなく「この型を使って」と指定することで出力が安定する。



3. 余計な仕事を引き受ける

日付の表示、合計の計算、色の判定 —— **プログラムで確実にできることはプログラムに任せる**。AIは「わかりやすい言葉への変換」「提案文の作成」など、AIにしかできないことに集中させる。

≡ AI-Driven Rules 振り返り（1ヶ月目の10ルール）

第1回・第2回講義で学んだ10個のルール。今回の報告自動化の文脈で、改めて振り返ります。

01 90点から100点の戦いに備えよ
残り10点に全エネルギーの9割が費やされる

02 理解できないものは作るな
「なぜこう動くのか」と問い続ける

03 自分の武器を自分で作れ
面倒な仕事を消す専用ツールを自分の手で

04 図解で認知コストを下げろ
理解するために描く、伝えるために描く

05 議論する前にプロトタイプを作れ
言葉で伝えれば必ず劣化する。作って見せる

06 AIが見る情報を整えろ
SSoTを厳守し、データを整え続ける

07 スキルを育て続けろ

AIの精度が上がり、できることの幅が広がる

08 抽象より模範解答を作れ

正解を1つ見せる方がAIの出力は格段によくなる

09 スピードを出すな 対話しろ

解像度が低いまま突っ走ると行き詰まる

10 1クリックを減らせ

操作を1つ減らす執着がツールの使い心地を変える

☑ まとめ——第3回講義で覚えておきたいこと

01 報告の自動化は「認知」を変える

報告が自動で届くようになると、作業時間が浮くだけでなく、認知が早くなり、意思決定の質が上がる。本当の価値は「起きなかった事故」「取り逃がさなかった売上」など、見えない成果にある。

02 どんなツールも4パーツで分解できる

トリガー（いつ動く）・ソース元（何のデータ）・処理する場所（どこで動かす）・届ける先（どこに届ける）。この4つの解像度を上げることが、ツール作りの第一歩。

03 3つのコツで品質を上げる

完成形の絵から作る → サブエージェントで磨く → プレをコードで止める。AIだけに頼らず、プログラムと組み合わせることで安定した品質のレポートが届くようになる。



今月の課題：面倒な報告を自動化するツールを作る

教科書の練習問題ではなく、明日から自分の仕事で使える武器作り。最高の学びは自分の武器を作ること。

次回グループセッションまでに:

- 必 完成イメージの「絵」を作る（どんなレポートが届くといいか）
- 推 4つのパーツに分類して設計を言語化する
- 可 3つのコツをどう生かすか説明できるとなご良い

AI-Driven School の図解ツールで作成